N+1 Query and Request Analysis

Scope and methodology

This is a source-level review of backend database access in `src/` and frontend data-loading in `frontend/public/js/`. Findings were traced through callers and routes to verify query/request growth. No database traffic was run and no production benchmark was performed. Priorities describe likely impact, not measured latency.

Confirmed N+1 and per-record fan-out

Priority	Finding and evidence	Growth and impact	Recommended direction
High	**Tournament projection rebuild** — `src/services/matchProjectionService.js:132-154`, called after referee event changes and phase completion in `src/services/refereeService.js:298-300`, `330-333`, `413-415`, `540`.	Every rebuild resets team/player projections, then by row, queries events separately for each finished match, and updates player stats individually per relevant event. Work grows with tournament teams, players, matches, and events. Simulation also rebuilds before and after finishing a match (`src/services/simulationService.js:233-244`).	Replace per-row resets and player-stat updates with set-based SQL; read events for all finished matches in one query and aggregate together. Consider incremental projection updates for affected matches/players, while preserving projection correctness.

Priority	Finding and evidence	Growth and impact	Recommended direction
High	**Per-team roster API fan-out** — `frontend/public/js/tournament.js:911-915`, `2263-2317`, `2339-2353`; endpoint `src/controllers/catalogController.js:534-552`	Teams & players, moderator vote-candidate loading, and leaderboards make one roster request per team. Requests are concurrent, but request count and backend DB work remain O(teams). Each roster request uses about four DB queries, or five for some private-tournament checks. The voting view loads all candidate rosters before checking whether any votes exist.	Add a tournament-scoped roster/candidate/leaderboard API response or batch-by-team endpoint, and cache/share it across tabs. Defer candidate loading until needed.
Medium	**Sequential nominee loading per vote** — `frontend/public/js/tournament.js:906-908`, `1033-1081`.	Once votes are fetched, a `for...of` loop awaits each vote's nominee endpoint before requesting the next. Opening the view adds one serialized API round trip per vote.	Return votes with nominees from one endpoint or fetch nominees concurrently with `Promise.all`.

Priority	Finding and evidence	Growth and impact	Recommended direction
Medium	**Per-player suspension synchronization** — `src/services/suspensionService.js:45-96`, reached from match details (`src/services/matchDetailsService.js:211-223`) and kickoff checks (`src/controllers/matchOpsController.js:86-90`).	When enabled, each roster player gets a prior-match query and often a card-event query, plus insert/update/delete writes. Concurrency is capped at eight players, limiting bursts but not O(players) query growth.	Fetch prior matches/events and existing suspensions for all relevant players in batches; derive suspensions in memory and batch writes. Preserve uniqueness and transaction protections.
Medium	**Standings queries per group** — `src/controllers/tournamentController.js:145-163, 354-375`.	`rankGroupTeams` runs for every group: one team query, one group-match query, and another finished-match query when head-to-head tiebreaking is enabled. Processing groups in batches of three bounds concurrency but not total queries (about G to 2G).	Load teams and required matches for every stage group with a small number of queries, then rank groups in memory.

Priority	Finding and evidence	Growth and impact	Recommended direction
Medium	**Per-team invite-code cleanup on tournament updates** — `PATCH /tournaments/:tournamentId` → `src/controllers/tournamentController.js:396-480`	The transaction loops over every team even when the patch is unrelated to registration settings, runs code cleanup and then, depending on mode, additional deletes or roster-count/code-list reads. Query count grows with team count.	Skip cleanup unless related settings change; where cleanup is needed, aggregate roster/code data and perform bulk deletes while retaining team locks and transaction semantics.
Low–Medium	**One impact-count query per selected stage** — `src/controllers/refereeController.js:101-140`	The impact endpoint concurrently performs one query per selected directly affected next-stage ID. Total DB work scales with selected stage count. Later reset/redraw actions are state-changing and intentionally sequential.	Fetch counts for all selected stages using one grouped query. Keep dependent reset/redraw operations ordered.

Related bulk-write fan-out

`src/services/drawService.js:587-617, 659-685` updates each assigned team individually and inserts each generated fixture individually. This is not a classic read N+1, but league draws make row-at-a-time writes proportional to team and fixture counts; fixture count grows approximately quadratically with group size. Bulk inserts and updates could reduce round trips.

Existing bounds and limitations

- Standings ranking runs in batches of three groups.
- Suspension synchronization runs in batches of eight players.
- These limits reduce peak concurrency, but do not eliminate per-entity query growth.
- The findings are based on code inspection; no current latency estimates are asserted. Validate priorities and improvement magnitude with request/query timing before selecting implementation work.